Κεφάλαιο 10

Σύνοψη
Καταρχάς γίνεται μια σύντομη ιστορική αναδρομή της ανάπτυξης του αντικειμενοστρεφούς μοντέλου. Στη συνέχεια παρουσιάζονται τα θεμελιώδη χαρακτηριστικά του αντικειμενοστρεφούς μοντέλου, η Ενσωμάτωση (Encapsulation), η Κληρονομικότητα (Inheritance), ο Πολυμορφισμός (Polymorphism) και η Αφαιρετικότητα (Abstraction). Συζητείται η αναλογία μεταξύ των θεμελιωδών χαρακτηριστικών και των προβλημάτων του φυσικού κόσμου και εξηγούνται οι τρόποι με τους οποίους το αντικειμενοστρεφές μοντέλο διευκολύνει στη μοντελοποίηση των προβλημάτων του φυσικού κόσμου.

Προαπαιτούμενη γνώση
Η εισαγωγική αυτή ενότητα προϋποθέτει γνώση του διαδικαστικού προγραμματισμού με Java.

Λέξεις κλειδιά
Αντικειμενοστρεφής προγραμματισμός (Object Oriented Programming), Διαδικαστικός Προγραμματισμός (Procedural Programming), Ενσωμάτωση (Encapsulation), Κληρονομικότητα (Inheritance), Πολυμορφισμός (Polymorphism), Αφαιρετικότητα (Abstraction).

10 Εισαγωγή στο Αντικειμενοστρεφές μοντέλο

Το αντικειμενοστρεφές μοντέλο αντλεί την καταγωγή του από την εφαρμογή Sketchpad [1] του Ivan Sutherland που δημιουργήθηκε στις αρχές της δεκαετίας του 1960. Η Sketchpad υποστήριζε κληρονομικότητα βασιζόμενη σε πρωτότυπα με πολλά κοινά στοιχεία με τη σημερινή Javascript και δυναμική διασύνδεση, βασικό χαρακτηριστικό των περισσοτέρων γλωσσών αντικειμενοστρεφούς προγραμματισμού, σήμερα. Την ίδια περίοδο έκανε την εμφάνισή της η AED-0, μια έκδοση της Algol που υποστήριζε την άμεση διασύνδεση δεδομένων και συναρτήσεων που αποτελεί θεμελιώδες χαρακτηριστικό του αντικειμενοστρεφούς μοντέλου. Το 1965 εμφανίστηκε η Simula [2], η πρώτη γλώσσα προγραμματισμού που στη συνέχεια αναγνωρίστηκε ευρέως ως αντικειμενοστρεφής. Ωστόσο, τον όρο «Αντικειμενοστρεφής Προγραμματισμός» επινόησε ο Alan Kay το 1967.

Τη δεκαετία του 1970 αναπτύχθηκε η Smalltalk [3] που εισήγαγε την έννοια των κλάσεων από τους Alan Kay, Dan Ingalls, Adele Goldberg και άλλους στην Xerox PARC.

Στα μέσα του 1980 αναπτύχθηκε η Objective-C που προσέθεσε στην C στοιχεία από την SmallTalk και αποτέλεσε μέχρι το 2014, οπότε αντικαταστάθηκε από την Swift, την κύρια γλώσσα προγραμματισμού για τα λειτουργικά macOS και iOS της Apple.

Ήδη από το 1979, ο Bjarne Stroustrup, άρχισε την ανάπτυξη μιας προέκτασης της C που ονόμασε C with classes, ενώ το 1982 άρχισε την ανάπτυξη της C++ [4], η αρχική έκδοση της οποίας βγήκε το 1985.

Τη δεκαετία του 1990, το αντικειμενοστρεφές μοντέλο έγινε το κυρίαρχο μοντέλο προγραμματισμού, καθώς οι αντικειμενοστρεφείς γλώσσες διαδόθηκαν ευρέως.

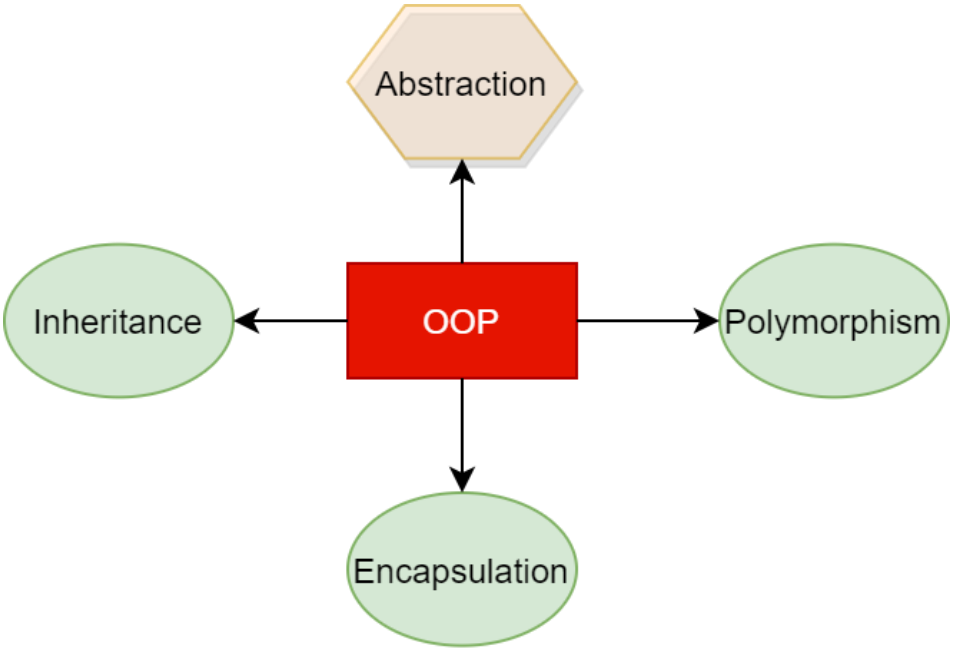
Η πρώτη έκδοση της Python κυκλοφόρησε το 1994 με δημιουργό τον Van Rossum. Με την εμφάνισή της, η Python ενσωματώνει τα βασικά χαρακτηριστικά του αντικειμενοστρεφούς μοντέλου τα οποία συνδυάζει με το λειτουργικό (functional) μοντέλο. Λίγο αργότερα, το 1996, κυκλοφόρησε και η πρώτη έκδοση της Java το 1996 από την Sun Microsystems.

Πέραν των προαναφερόμενων γλωσσών, εμφανίστηκαν και αρκετές άλλες ενώ χαρακτηριστικά του αντικειμενοστρεφούς μοντέλου, προστέθηκαν σε αρκετές προϋπάρχουσες.

Πρόσφατα, πολλές αντικειμενοστρεφείς γλώσσες, συμπεριλαμβανομένης της Java, ενσωματώνουν χαρακτηριστικά του λειτουργικού (functional) προγραμματισμού, ενισχύοντας έτσι την αποτελεσματικότητά τους.

10.1 Τα θεμελιώδη χαρακτηριστικά του Αντικειμενοστρεφούς Προγραμματισμού

Το αντικειμενοστρεφές μοντέλο σχεδιάστηκε ώστε να εξυπηρετεί δύο βασικούς στόχους. Ο πρώτος αφορά στη διευκόλυνση της μοντελοποίησης των προβλημάτων του πραγματικού κόσμου και ο δεύτερος στον έλεγχο των λαθών που όχι σπάνια υπεισέρχονται στους κώδικες εφαρμογών.
Τους στόχους του, το αντικειμενοστρεφές μοντέλο επιτυγχάνει βασιζόμενο στα θεμελιώδη χαρακτηριστικά του: την Ενσωμάτωση (Encapsulation), την Κληρονομικότητα (Inheritance) και τον Πολυμορφισμό (Polymorphism). Στο σχήμα 10.1, παρουσιάζονται τα θεμελιώδη χαρακτηριστικά του αντικειμενοστρεφούς μοντέλου. Στο σχήμα συμπεριλαμβάνεται και η Αφαιρετικότητα (Abstraction) που παρουσιάζεται παρακάτω σε αυτήν την ενότητα.

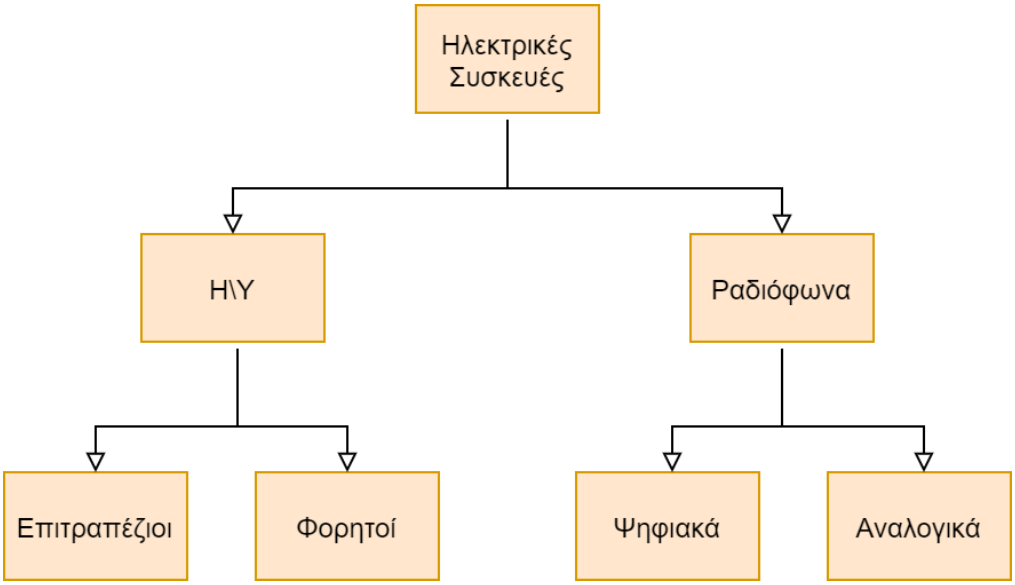


Σχήμα 10.1 *Τα θεμελιώδη χαρακτηριστικά του αντικειμενοστρεφούς μοντέλου*

Ο πραγματικός κόσμος αποτελείται από αντικείμενα, π.χ. ένας Ηλεκτρονικός Υπολογιστής, ένα βιβλίο, μια καρέκλα. Τα αντικείμενα συχνά διαθέτουν ιδιότητες και επιτελούν λειτουργίες. Για παράδειγμα, ένας Ηλεκτρονικός Υπολογιστής διαθέτει ιδιότητες όπως η χωρητικότητα της μνήμης RAM, ο τύπος του επεξεργαστή του κ.ά. Επίσης, επιτελεί λειτουργίες, όπως εκκίνηση λειτουργίας, τερματισμός, κ.ά. Το αντικειμενοστρεφές μοντέλο μας δίνει τη δυνατότητα να ορίσουμε οντότητες μέσα στον κώδικά μας που ενσωματώνουν ιδιότητες και λειτουργίες και να αναπαραστήσουμε έτσι τα αντικείμενα του πραγματικού κόσμου με ευκολία. Πρόκειται για την πιο βασική αρχή του αντικειμενοστρεφούς προγραμματισμού, την Ενσωμάτωση. Λεπτομέρειες για την Ενσωμάτωση θα βρείτε στην ενότητα 11.

Τα αντικείμενα του πραγματικού κόσμου είναι οργανωμένα σε κατηγορίες. Για παράδειγμα, ο δικός μου υπολογιστής ανήκει στην κατηγορία των επιτραπέζιων υπολογιστών, ο φορητός υπολογιστής της συζύγου μου ανήκει στην κατηγορία των φορητών υπολογιστών, το ραδιόφωνό που έχουμε στο σαλόνι ανήκει στην κατηγορία των αναλογικών ραδιοφώνων και αυτό που έχουμε στην κρεβατοκάμαρα στην κατηγορία των ψηφιακών ραδιοφώνων. Στα πλαίσια του αντικειμενοστρεφούς μοντέλου, οι κατηγορίες αντικειμένων αναφέρονται συνήθως με τον όρο κλάσεις (classes). Έτσι, έχουμε την κλάση των επιτραπέζιων υπολογιστών και την κλάση των φορητών υπολογιστών, την κλάση των αναλογικών ραδιοφώνων και την κλάση των ψηφιακών ραδιοφώνων. Προσέξτε όμως πως οι επιτραπέζιοι και οι φορητοί υπολογιστές συνιστούν υποκατηγορίες ή υποκλάσεις (subclasses) της κλάσης των ηλεκτρονικών υπολογιστών. Αντίστοιχα, η κλάση των ηλεκτρονικών υπολογιστών είναι υπερκλάση (superclass) της κλάσης των επιτραπέζιων και της κλάσης των φορητών υπολογιστών. Παρόμοια, τα αναλογικά και τα ψηφιακά ραδιόφωνα αποτελούν υποκλάσεις της γενικότερης κλάσης των ραδιοφώνων. Επιπλέον, τόσο τα ραδιόφωνα όσο και οι ηλεκτρονικοί υπολογιστές είναι υποκλάσεις της κλάσης των ηλεκτρικών συσκευών.

Γενικότερα, τα αντικείμενα του πραγματικού κόσμου οργανώνονται σε κλάσεις οι οποίες σχετίζονται μεταξύ τους με ιεραρχικές σχέσεις. Στο σχήμα 10.2 απεικονίζονται οι ιεραρχικές σχέσεις των κλάσεων που αναφέραμε ως παραδείγματα.



Σχήμα 10.2 *Ιεραρχικές σχέσεις μεταξύ των κλάσεων*

Οι ηλεκτρικές συσκευές διαθέτουν μια λειτουργία που τους επιτρέπει να συνδέονται ή να αποσυνδέονται στο ηλεκτρικό ρεύμα. Τη λειτουργία αυτή όμως διαθέτουν και οι Η\Υ και τα ραδιόφωνα καθώς τόσο οι Η\Υ όσο και τα ραδιόφωνα είναι ηλεκτρικές συσκευές. Το ίδιο ισχύει για τις κλάσεις των επιτραπέζιων και φορητών υπολογιστών, των ψηφιακών και των αναλογικών ραδιοφώνων. Γενικότερα, μια λειτουργία που ορίζεται σε μια υπερκλάση αποτελεί επίσης λειτουργία κάθε υποκλάσης της. Το αντικειμενοστρεφές μοντέλο μέσα από τη δεύτερη θεμελιώδη αρχή του, την Κληρονομικότητα, μας δίνει τη δυνατότητα να ορίζουμε μια ιδιότητα ή λειτουργία σε μια κλάση και στη συνέχεια η λειτουργία/ιδιότητα κληρονομείται αυτόματα σε όλες τις υποκλάσεις της. Το χαρακτηριστικό αυτό διευκολύνει την εφαρμογή της επαναχρησιμοποίησης κώδικα (code reusability) και διαμορφώνει το μοντέλο για την εφαρμογή του τρίτου θεμελιώδους χαρακτηριστικού του, του Πολυμορφισμού. Λεπτομέρειες για την Κληρονομικότητα θα βρείτε στην ενότητα 13.

Όπως αναφέρθηκε ήδη, ο υπολογιστής μου είναι επιτραπέζιος, επομένως ανήκει στην κλάση των επιτραπέζιων υπολογιστών. Ταυτόχρονα όμως είναι ένας (is a) Η\Υ αλλά και μια ηλεκτρική συσκευή. Σε αντιστοιχία με τον πραγματικό κόσμο, το αντικειμενοστρεφές μοντέλο μας δίνει τη δυνατότητα να δημιουργήσουμε ένα αντικείμενο τύπου επιτραπέζιου υπολογιστή και ανάλογα με τις ανάγκες του κώδικά μας να το διαχειριζόμαστε σαν Η\Υ ή σαν ηλεκτρική συσκευή, δηλαδή σαν να είναι τύπου μιας υπερκλάσης. Αυτή η δυνατότητα συνιστά τη βάση του Πολυμορφισμού και είναι πολύ σημαντική όπως θα δούμε στην ενότητα 13 που εξετάζουμε αναλυτικά τον πολυμορφισμό.

Πολλοί συγγραφείς θεωρούν ένα τέταρτο θεμελιώδες χαρακτηριστικό του αντικειμενοστρεφούς προγραμματισμού, την Αφαιρετικότητα (Abstraction) [1,4]. Αφαιρετικότητα σημαίνει απόκρυψη των μη αναγκαίων πληροφοριών από τον χρήστη του κώδικα. Ωστόσο, άλλοι συγγραφείς δεν θεωρούν την αφαιρετικότητα ως θεμελιώδες χαρακτηριστικό του αντικειμενοστρεφούς μοντέλου [2,3]. Παρότι υπάρχει καθολική συμφωνία πως η αφαιρετικότητα είναι ένα χαρακτηριστικό του αντικειμενοστρεφούς προγραμματισμού, προκύπτει διχογνωμία για το κατά πόσο αποτελεί θεμελιώδες γνώρισμά του. Οι συγγραφείς που δεν θεωρούν την αφαιρετικότητα ως θεμελιώδες χαρακτηριστικό, επιχειρηματολογούν υποστηρίζοντας πως αυτή συναντάται και στο διαδικαστικό μοντέλο επάνω στο οποίο στηρίζεται το αντικειμενοστρεφές. Επίσης, κάθε πληροφοριακό σύστημα, σωστά οργανωμένο, χαρακτηρίζεται από κάποιο βαθμό αφαιρετικότητας. Επιπλέον, παρότι η αφαιρετικότητα διαθέτει και τους δικούς της ιδιαίτερους μηχανισμούς σε αρκετές γλώσσες και στην Java, είναι ταυτόχρονα αναγκαίο αποτέλεσμα της ενσωμάτωσης, της κληρονομικότητας και του πολυμορφισμού. Με αυτά τα δεδομένα, τείνουν να θεωρούν την αφαιρετικότητα ως παράγωγο παρά ως θεμελιώδες γνώρισμα.

Βιβλιογραφία

[1] “Sketchpad,” Wikipedia. Dec. 28, 2020. Accessed: Feb. 10, 2021. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Sketchpad&oldid=996705015

[2] J. R. Holmevik, “Compiling SIMULA: a historical study of technological genesis,” IEEE Annals of the History of Computing, vol. 16, no. 4, pp. 25–37, Winter 1994, doi: 10.1109/85.329756.

[3] “The Early History of Smalltalk,” Jul. 10, 2008. https://web.archive.org/web/20080710144930/http://gagne.homedns.org/~tgagne/contrib/EarlyHistoryST.html (accessed Feb. 10, 2021).

[4] “Stroustrup: The C++ Programming Language.” https://www.stroustrup.com/1st.html (accessed Feb. 10, 2021).

Κεφάλαιο 11

Σύνοψη
Σε αυτήν την ενότητα παρουσιάζεται η Ενσωμάτωση, το θεμελιώδες χαρακτηριστικό του αντικειμενοστρεφούς μοντέλου με βάση το οποίο οι συναρτήσεις συνδέονται άμεσα με τα δεδομένα στα οποία επενεργούν. Εξηγούνται αναλυτικά τα πλεονεκτήματα της Ενσωμάτωσης σχετικά με τη μοντελοποίηση του φυσικού κόσμου και τον περιορισμό των λαθών στις εφαρμογές. Παρουσιάζονται και εξηγούνται οι αναγκαίες συναρτήσεις μια κλάσης σύμφωνα με το μοντέλο της Java.

Προαπαιτούμενη γνώση
Το διαδικαστικό μοντέλο προγραμματισμού με Java

Λέξεις κλειδιά
Ενσωμάτωση (Encapsulation), αντικείμενο (object), κλάση (class), στιγμιότυπο (instance), μεταβλητή στιγμιότυπου (instance variable), συνάρτηση στιγμιότυπου (instance function), μέθοδος (method), δημιουργός (constructor), αναγνώστης (getter), ρυθμιστής (setter), προκαθορισμένη προσπέλαση ή προσπέλαση πακέτου (default access ή package access)

11 Ενσωμάτωση

Ας υποθέσουμε ότι θέλουμε να αναπτύξουμε μια εφαρμογή στην οποία απαιτούνται υπολογισμοί με διάφορα γεωμετρικά σχήματα, π.χ. χρειάζεται να υπολογίσουμε την περίμετρο ενός ορθογώνιου παραλληλόγραμμου, το εμβαδόν ενός κύκλου, κ.ά. Στο πλαίσιο του διαδικαστικού μοντέλου θα αναπτύξουμε κατάλληλη βιβλιοθήκη συναρτήσεων και στην εφαρμογή μας θα καλούμε τις συναρτήσεις της βιβλιοθήκης. Ο τρόπος για να υλοποιήσουμε μια τέτοια βιβλιοθήκη στην Java είναι η ανάπτυξη μιας ή περισσοτέρων κλάσεων που θα περιλαμβάνουν στατικές συναρτήσεις.

Στον κώδικα 11.1, παρουσιάζουμε μια τέτοια κλάση. Για απλοποίηση θεωρούμε ότι μας ενδιαφέρει μόνο το εμβαδόν, η περίμετρος και η διαγώνιος του ορθογώνιου παραλληλόγραμμου.

```
public class Lib {

    public static double rectArea(double length, double width) {
        return length * width;
    }

    public static double rectPerimeter(double length, double width) {
        return 2 * (length + width);
    }

    public static double rectDiagonal(double length, double width) {
        return Math.sqrt(Math.pow(length, 2) + Math.pow(width, 2));
    }
}
```

Κώδικας 11.1 *Βιβλιοθήκη στατικών συναρτήσεων γεωμετρικών υπολογισμών*

Στη συνέχεια στον κώδικα-πελάτη (client code), δηλαδή στην εφαρμογή μας θα χρησιμοποιήσουμε τις συναρτήσεις της Lib για να κάνουμε τους απαιτούμενους υπολογισμούς, όπως φαίνεται στον κώδικα 11.2. Σημειώστε πως για να χρησιμοποιήσετε την Lib θα πρέπει να την κάνετε import.

```
public class App {

    public static void main(String[] args) {
        double rectA = 6, rectB = 8;
        System.out.println(rectArea(rectA, rectB));
        System.out.println(rectPerimeter(rectA, rectB));
        System.out.println(rectDiagonal(rectA, rectB));
```

```
        }
    }
```

Κώδικας 11.2 *Εφαρμογή που χρησιμοποιεί τις συναρτήσεις της Lib*

Όπως δείχνει ο κώδικας 11.2, η εφαρμογή καλεί τις συναρτήσεις βιβλιοθήκης, περνώντας ως πραγματικές παραμέτρους τις μεταβλητές rectA και rectB που αναπαριστούν το μήκος και το πλάτος ενός ορθογώνιου. Ο μοναδικός έλεγχος από τον μεταγλωττιστή σε αυτήν τη φάση αφορά στη συμβατότητα τύπου μεταξύ πραγματικών και τυπικών παραμέτρων. Τι θα συμβεί όμως αν ο προγραμματιστής της εφαρμογής περάσει ως παράμετρο μια μεταβλητή τύπου double που δεν αναπαριστά ούτε μήκος, ούτε πλάτος ορθογώνιου; Τι θα συμβεί αν περάσει ως παραμέτρους μεταβλητές που αναπαριστούν το μήκος μιας πλευράς ενός ορθογώνιου και το μήκος πλευράς άλλου ορθογώνιου; Τι θα συμβεί αν περάσει μια αναρχικοποίητη μεταβλητή; Σε όλες αυτές τις περιπτώσεις θα έχει εισαχθεί ένα λάθος χρόνου εκτέλεσης στην εφαρμογή. Μπορεί να σκεφτείτε πως το πρόβλημα δεν είναι πολύ σοβαρό, καθώς έχουμε όλες κι όλες 3 μεταβλητές, οι δύο εκφράζουν το μήκος και το πλάτος ενός ορθογώνιου και η τρίτη, η radius, ενδεχομένως την ακτίνα ενός κύκλου. Επομένως είναι κάπως δύσκολο ο προγραμματιστής εφαρμογών να κάνει λάθος. Κάτι τέτοιο φαίνεται να ισχύει για το παράδειγμά μας. Σκεφτείτε όμως έναν ρεαλιστικό κώδικα, με χιλιάδες γραμμές, με χιλιάδες μεταβλητές στον οποίο κάποιες μεταβλητές υπολογίζονται κατά τον χρόνο εκτέλεσης. Ένα τέτοιο περιβάλλον αυξάνει σημαντικά την πιθανότητα λαθών τα οποία όμως είναι πολύ σοβαρά και συχνά οδηγούν σε απρόβλεπτες συνέπειες με μεγάλο κόστος. Επομένως, δεν είναι καλή ιδέα, η αποφυγή τους να επαφίεται αποκλειστικά στην ευθύνη του προγραμματιστή εφαρμογών. Ωστόσο, το διαδικαστικό μοντέλο δεν μπορεί να δώσει ικανοποιητικές απαντήσεις στο πρόβλημά μας. Αυτό οφείλεται στο ότι το μοντέλο διατηρεί χωριστά τις λειτουργίες, δηλαδή τις συναρτήσεις από τα δεδομένα στα οποία αυτές επενεργούν. Ικανοποιητική απάντηση στα ερωτήματα μας έρχεται να δώσει το αντικειμενοστρεφές μοντέλο με την Ενσωμάτωση (Encapsulation).

Με την Ενσωμάτωση, συναρτήσεις και δεδομένα συνδυάζονται σε μια οντότητα, το λεγόμενο αντικείμενο (object). Σε σχέση με το παράδειγμά μας, μπορούμε να δημιουργούμε αντικείμενα που αναπαριστούν ορθογώνια στα οποία τα δεδομένα, δηλαδή το μήκος των πλευρών συνδέεται με τις συναρτήσεις υπολογισμού της περιμέτρου, του εμβαδού και της διαγωνίου. Σε αυτήν την περίπτωση δεν απαιτείται να περάσουμε ως παραμέτρους στις εν λόγω συναρτήσεις μεταβλητές που αναπαριστούν τα μήκη των πλευρών, οπότε αυτομάτως περιορίζεται ο κίνδυνος λαθών. Προκειμένου όμως να είμαστε σε θέση να δημιουργήσουμε τέτοια αντικείμενα θα πρέπει πρώτα να ορίσουμε την κλάση τους. Στον κώδικα 11.3 δίνεται ένας αρχικός ορισμός της κλάσης των ορθογώνιων παραλληλόγραμμων.

```
public class Rectangle {

    public double length, width;

    public double area() {
        return length * width;
    }

    public double perimeter() {
        return 2 * (length + width);
    }

    public double diagonal() {
        return Math.sqrt(Math.pow(length, 2) + Math.pow(width, 2));
    }
}
```

Κώδικας 11.3 *Πρώτη έκδοση της κλάσης Rectangle*

Η κλάση ορίζεται με τη χρήση της λέξης-κλειδί class ακολουθούμενης από το όνομα που δίνουμε στην κλάση. Στη δική μας περίπτωση διαλέξαμε το όνομα Rectangle. Προσέξτε πως οι μεταβλητές length και width είναι μη στατικές (non-static). Οι μεταβλητές αυτές ονομάζονται και μεταβλητές στιγμιότυπου (instance variables). Επίσης οι μη στατικές συναρτήσεις area, perimeter και diagonal ονομάζονται συναρτήσεις στιγμιότυπου (instance functions). Επιπλέον, οι συναρτήσεις μιας κλάσης ονομάζονται και μέθοδοι (methods). Ας δούμε όμως πώς δημιουργούμε και αξιοποιούμε τα αντικείμενα της κλάσης Rectangle.

```
public class App2 {

    public static void main(String[] args) {
        Rectangle r = new Rectangle();
        r.length = 2d;
        r.width = 2d;
        System.out.println(r);
        System.out.println(r.area());
        System.out.println(r.perimeter());
        System.out.println(r.diagonal());
    }
}
```

Κώδικας 11.4 *Εφαρμογή που αξιοποιεί την κλάση Rectangle*

Στην πρώτη γραμμή της main δηλώνουμε μια μεταβλητή r τύπου Rectangle. Παράλληλα δημιουργούμε ένα Rectangle με χρήση της λέξης-κλειδί new. Επομένως, η μεταβλητή r είναι μεταβλητή αναφοράς. Το new συνοδεύεται από το Rectangle() που αποτελεί τον προκαθορισμένο δημιουργό (default constructor) της κλάσης. Για τους δημιουργούς (constructors) συζητάμε αναλυτικά παρακάτω σε αυτήν την ενότητα. Για την ώρα αρκεί να γνωρίζουμε πως η κλήση στην πρώτη γραμμή της main δημιουργεί ένα αντικείμενο Rectangle και εκχωρεί τη διεύθυνσή του στη μεταβλητή r. Το αντικείμενο (object) που δημιουργείται ονομάζεται και στιγμιότυπο (instance). Κατά τη δημιουργία του στιγμιότυπου, κάθε bit μη στατικής μεταβλητής αρχικοποιείται στο 0. Επομένως, τα πεδία length και width αρχικοποιούνται αυτόματα στην τιμή 0d. Στη συνέχεια, η εφαρμογή προσπελαύνει τα length και width και τους δίνει τιμές. Όπως φαίνεται, η προσπέλαση γίνεται με χρήση του τελεστή τελεία (dot operator) και της μεταβλητής r που αναφέρεται στο συγκεκριμένο στιγμιότυπο. Ας τονιστεί εδώ πως κάθε στιγμιότυπο διαθέτει δικές του μεταβλητές στιγμιότυπου. Αν για παράδειγμα δημιουργήσουμε ένα ακόμη Rectangle, ας πούμε r1, τότε για τις μεταβλητές length και width του r1, θα δεσμευτεί επιπλέον χώρος στη μνήμη για δύο double. Επομένως κάθε στιγμιότυπο Rectangle "γνωρίζει" τις διαστάσεις του και επάνω σε αυτές επενεργούν οι συναρτήσεις του. Είναι φανερό πως αυτή η οργάνωση συνδέει άμεσα τις συναρτήσεις με τις μεταβλητές ενός στιγμιότυπου, απαλλάσσοντας έτσι τον προγραμματιστή της εφαρμογής από αυτήν την ευθύνη. Αυτός είναι ο λόγος που οι συναρτήσεις area, perimeter και diagonal δεν λαμβάνουν παραμέτρους όπως οι αντίστοιχες στατικές συναρτήσεις rectArea, rectPerimeter και rectDiagonal.

Παρόλα αυτά, ο κώδικάς μας έτσι όπως έχει, συνεχίζει να περιλαμβάνει σημαντικές αδυναμίες. Προσέξτε πως η εφαρμογή δίνει τιμές στις διαστάσεις ενός Rectangle. Τι θα γίνει αν για παράδειγμα δοθεί αρνητική τιμή σε κάποια από τις διαστάσεις, π.χ. length=2, width=-2; Θα έχουμε δημιουργήσει ένα ορθογώνιο που δεν θα είναι σωστό. Στη συνέχεια, αν ζητήσουμε το εμβαδόν του, θα λάβουμε την αρνητική τιμή -4 που είναι χωρίς νόημα και αν ζητήσουμε την περίμετρό του θα λάβουμε την τιμή 0, επίσης λανθασμένη. Χρησιμοποιώντας τον προκαθορισμένο δημιουργό έχουμε μεταθέσει στην εφαρμογή την ευθύνη να δώσει τιμές στα length και width. Η δημιουργία όμως αντικειμένων της κλάσης γίνεται πιο ασφαλής αν ολοκληρώνεται από την ίδια την κλάση. Η κλάση γνωρίζει τους περιορισμούς στους οποίους υπόκεινται τα μέλη της, ενώ η εφαρμογή δεν είναι αναγκαίο να τους γνωρίζει.

11.1 Οι δημιουργοί

Ο έλεγχος της δημιουργίας των αντικειμένων από την κλάση γίνεται με τη βοήθεια των δημιουργών (constructors) της κλάσης. Ένας δημιουργός είναι μια συνάρτηση μέλος της κλάσης που έχει όνομα ίδιο με της κλάσης και δεν έχει τιμή επιστροφής. Μέσα στον δημιουργό, η κλάση μπορεί να εκτελέσει τους απαραίτητους ελέγχους ώστε να διασφαλισθεί ότι τα στιγμιότυπα που δημιουργούνται είναι σωστά. Στον κώδικα 11.5 παρουσιάζουμε έναν δημιουργό για την κλάση Rectangle.

```
public Rectangle(double len, double wid) {
    if (len <= 0 || wid <= 0) {
        throw new IllegalArgumentException();
    }
    length = len;
    width = wid;
```

```
}
```

Κώδικας 11.5 *Δημιουργός της κλάσης Rectangle*

Αυτός ο δημιουργός λαμβάνει δύο παραμέτρους, η μια αντιπροσωπεύει το μήκος και η άλλη το πλάτος του ορθογώνιου που δημιουργείται κατά την κλήση του. Ο δημιουργός ελέγχει πρώτα αν οι τιμές των παραμέτρων είναι αποδεκτές. Αν όχι, προχωράει στη διαχείριση του λάθους. Στη δική μας περίπτωση η διαχείριση αυτή συνίσταται στην παραγωγή κατάλληλης εξαίρεσης που τερματίζει την εφαρμογή παράγοντας αντίστοιχο μήνυμα. Αν όμως οι παράμετροι περάσουν τον έλεγχο επιτυχώς, τότε οι τιμές τους αντιγράφονται στις μεταβλητές στιγμιότυπου από τον κώδικα που ακολουθεί και συντελείται η δημιουργία ενός ορθού αντικειμένου.

Η κλήση του νέου δημιουργού γίνεται κατά τη δημιουργία των στιγμιότυπων με κώδικα της μορφής:

```
Rectangle r = new Rectangle(2, 5);
Rectangle t = new Rectangle(3, 3);
```

Εδώ δημιουργούμε δύο ορθογώνια, το ένα έχει μήκος 2 και πλάτος 5 και το άλλο είναι τετράγωνο με μήκος πλευράς 3. Ιδιαίτερη προσοχή θα πρέπει να δοθεί στο γεγονός ότι η πρόσθεση ενός δημιουργού στην κλάση ακυρώνει αυτόματα τον προκαθορισμένο δημιουργό. Με άλλα λόγια ο κώδικας 11.4 δεν μεταγλωττίζεται πλέον. Ο προκαθορισμένος δημιουργός είναι αναγκαστικά δημιουργός χωρίς παραμέτρους. Αν επιθυμούμε ένα δημιουργό χωρίς παραμέτρους, τότε θα πρέπει να ορίσουμε σαφώς έναν τέτοιο μέσα στην κλάση. Για παράδειγμα:

```
public Rectangle() {
     length = 2;
     width = 2;
}
```

Ο κώδικας 11.4 μεταγλωττίζεται τώρα. Ωστόσο, το ορθογώνιο που δημιουργείται δεν έχει μήκος και πλάτος 0 αλλά 2. Γενικά μπορούμε να προσθέσουμε όσους δημιουργούς απαιτούνται σε μια κλάση αρκεί να ακολουθούν τους κανόνες υπερφόρτωσης συναρτήσεων (function overloading), δηλαδή να διαφοροποιείται η ταυτότητά τους (signature). Για παράδειγμα, μπορούμε να προσθέσουμε έναν δημιουργό για να κατασκευάζουμε τετράγωνα. Δεδομένου ότι το μήκος ενός τετραγώνου είναι ίσο με το πλάτος του, ένας τέτοιος δημιουργός θα χρειάζεται μια παράμετρο.

```
public Rectangle(double len) {
     if (len <= 0) {
          throw new IllegalArgumentException();
     }
     length = len;
     width = len;
}
```

Εφόσον προσθέσουμε έναν τέτοιο δημιουργό στην κλάση, η κλήση Rectangle r=new Rectangle(5), αναφέρεται στον δημιουργό που έχει μια παράμετρο και κατασκευάζει ένα τετράγωνο με πλευρά μήκους 5.

11.2 Αναγνώστες και Ρυθμιστές

Η νέα μορφή της Rectangle μας εξασφαλίζει πως τα αντικείμενα, εφόσον δημιουργούνται, είναι ορθά. Παρόλα αυτά, ο κώδικάς μας έχει πάλι σοβαρές αδυναμίες. Παρότι έχουμε εγγυηθεί την ορθή δημιουργία των αντικειμένων, δεν τα προστατεύουμε καθόλου μετά τη δημιουργία τους. Η εφαρμογή μπορεί να προσπελάσει τα ευαίσθητα μέλη της κλάσης και να εκχωρήσει λανθασμένες τιμές μέσω του τελεστή τελεία (dot operator). Ένας κώδικας της μορφής

```
Rectangle r = new Rectangle(2, 5);
r.length=-5;
```

καταστρέφει την ακεραιότητα του αντικειμένου r.

Λύση στο πρόβλημα μας δίνει η δήλωση ως private των μεταβλητών length και width, οπότε ο κώδικας πελάτη δεν μπορεί να τις προσπελάσει. Με άλλα λόγια, κώδικας της μορφής r.length=-5 έξω από την κλάση δεν μεταγλωττίζεται εφόσον η length έχει χαρακτηριστεί private. Επομένως, έχουμε εγγυηθεί πως το αντικείμενο είναι ορθό κατά τη δημιουργία του και δεν μπορεί να καταστρέψει την ορθότητά του ο χρήστης της κλάσης. Ωστόσο, μπορεί να θέλουμε να παραχωρήσουμε πρόσβαση στον χρήστη ώστε να μπορεί να διαβάσει τις τιμές ενός private μέλους. Τον ρόλο αυτό έχουν αναλάβει ειδικές συναρτήσεις της κλάσης γνωστές ως αναγνώστες (getters). Επίσης, είναι πιθανό να θέλουμε να επιτρέψουμε στον κώδικα πελάτη να γράψει ένα private μέλος υπό τον έλεγχο της κλάσης. Τον ρόλο αυτό αναλαμβάνουν οι ρυθμιστές (setters) της κλάσης. Στη συνέχεια παρουσιάζεται η κλάση Rectangle με private μεταβλητές στιγμιότυπου, getters και setters.

```
public class Rectangle {

    private double length, width;

    public Rectangle(double len, double wid) {
        if (len <= 0 || wid <= 0) {
            throw new IllegalArgumentException();
        }
        length = len;
        width = wid;
    }

    public Rectangle(double len) {
        if (len <= 0) {
            throw new IllegalArgumentException();
        }
        length = len;
        width = len;
    }

    public Rectangle() {
        length = 2;
        width = 2;
    }

    public double area() {
        return length * width;
    }

    public double perimeter() {
        return 2 * (length + width);
    }

    public double diagonal() {
        return Math.sqrt(Math.pow(length, 2) + Math.pow(width, 2));
    }

    public double getLength() {
        return length;
    }

    public void setLength(double len) {
        if (len <= 0) {
            throw new IllegalArgumentException();
        }
        length = len;
    }
```

```
        public double getWidth() {
            return width;
        }

        public void setWidth(double wid) {
            if (wid <= 0) {
                throw new IllegalArgumentException();
            }
            width = wid;
        }
    }
```

Κώδικας 11.6 *Η κλάση Rectangle με ιδιωτικές μεταβλητές, getters και setters*

Ο κώδικας 11.7 παρουσιάζει μια εφαρμογή που αξιοποιεί την κλάση Rectangle όπως έχει διαμορφωθεί μέχρι στιγμής.

```
public class App {

    public static void main(String[] args) {
        Rectangle r = new Rectangle(2, 5);
        Rectangle t = new Rectangle(3);
        r.setLength(5);
        System.out.println(t.getLength()+" "+t.getWidth());
        System.out.println(r.area());
        System.out.println(r.perimeter());
        System.out.println(r.diagonal());
      }
}
```

Κώδικας 11.7 *Εφαρμογή που αξιοποιεί την κλάση Rectangle με ασφάλεια*

Όπως βλέπουμε, ο χρήστης της κλάσης μπορεί να δημιουργήσει αντικείμενα Rectangle χρησιμοποιώντας έναν από τους διαθέσιμους δημιουργούς, να διαβάσει και να γράψει με ασφάλεια το μήκος και το πλάτος ενός τέτοιου αντικειμένου και να υπολογίσει το εμβαδόν, την περίμετρο και το μήκος της διαγωνίου χωρίς να αντιμετωπίσει κανένα από τα προβλήματα του διαδικαστικού μοντέλου όπως αυτά περιεγράφηκαν προηγουμένως σε αυτήν την ενότητα.

Ας σημειωθεί εδώ πως οι αναγνώστες ονομάζονται με το πρόθεμα get ακολουθούμενο από το όνομα του μέλους στο οποίο αναφέρονται. Σχηματίζεται έτσι σύνθετη λέξη και ισχύει η σύμβαση πως στα αναγνωριστικά (identifiers) που συνιστούν σύνθετες λέξεις, κάθε συνθετικό πλην του πρώτου θα πρέπει να αρχίζει από κεφαλαίο, π.χ. getLenth(). Αντίστοιχες συμβάσεις ισχύουν και για τους ρυθμιστές, μόνο που εδώ το πρόθεμα get αντικαθίσταται από το πρόθεμα set. Εξαίρεση σε αυτόν τον κανόνα αποτελούν οι αναγνώστες των μεταβλητών boolean στα ονόματα των οποίων αντί get έχουμε το πρόθεμα is.

11.3 Προσδιοριστές Προσπέλασης

Όπως βλέπουμε στον κώδικα 11.6, έχουμε προσδιορίσει την κλάση ως public. Αυτό σημαίνει ότι μπορεί να χρησιμοποιηθεί από οποιονδήποτε, δηλαδή από οποιαδήποτε κλάση στο ίδιο ή σε άλλο πακέτο και φυσικά από την ίδια την κλάση Rectangle. Επίσης ως public έχουν προσδιοριστεί οι δημιουργοί και τα υπόλοιπα μέλη-συναρτήσεις της κλάσης. Επομένως, επιτρέπεται η προσπέλασή τους από κάθε χρήστη της κλάσης είτε βρίσκεται στο ίδιο ή σε άλλο πακέτο. Η πρόσβαση που καθορίζεται από την απουσία προσδιοριστή προσπέλασης, ονομάζεται προκαθορισμένη προσπέλαση ή προσπέλαση πακέτου (default access ή package access). Σύμφωνα με αυτήν, η πρόσβαση στα μέλη της κλάσης επιτρέπεται μόνο από κλάσεις που ορίζονται στο ίδιο πακέτο. Έτσι αν ορίσουμε τον ρυθμιστή του πλάτους ως

```
    void setWidth(double wid) {
        if (wid<=0)
            throw new IllegalArgumentException();
        width = wid;
```

```
}
```

τότε μόνο οι κλάσεις που ανήκουν στο ίδιο πακέτο με την Rectangle θα μπορούν να τον καλέσουν και άρα μόνο αυτές θα μπορούν να μεταβάλουν το πλάτος ενός ορθογώνιου μετά τη δημιουργία του.
Τα μέλη-δεδομένα έχουν χαρακτηριστεί ως private. Αυτό σημαίνει πως επιτρέπεται η προσπέλασή τους μόνο μέσα στην κλάση Rectangle. Αν ωστόσο κάποιος χρήστης της κλάσης επιθυμεί να διαβάσει ή να γράψει τα μέλη αυτά μπορεί να χρησιμοποιήσει τους δημόσιους ρυθμιστές και αναγνώστες.
Τέλος, υπάρχει και ο προσδιοριστής protected που σχετίζεται με την κληρονομικότητα και παρουσιάζεται στην ενότητα 13.1.2.

11.4 Η λέξη κλειδί this

Προσέξτε πως οι μη στατικές συναρτήσεις καλούνται αναγκαστικά από ένα στιγμιότυπο της κλάσης. Μερικές φορές μέσα στον ορισμό μιας μη στατικής συνάρτησης είναι δυνατόν να χρειάζεται να αναφερθούμε στο στιγμιότυπο που την καλεί. Βεβαίως η συνάρτηση μπορεί να κληθεί από πολλά διαφορετικά στιγμιότυπα, κανένα εκ των οποίων δεν είναι γνωστό κατά τον χρόνο που αναπτύσσουμε τη συνάρτηση. Για τον λόγο αυτόν, η Java περνάει μια αναφορά στο στιγμιότυπο που κάλεσε τη συνάρτηση μέσω της λέξης-κλειδί this.

Σημειώστε πως επειδή οι στατικές συναρτήσεις δεν καλούνται υποχρεωτικά από αντικείμενο της κλάσης, δεν επιτρέπεται η χρήση του this μέσα σε στατική συνάρτηση.

Ακολουθεί παράδειγμα χρήσης της λέξης-κλειδί this.

```
public void setLength(double length) {
     if (length<=0)
          throw new IllegalArgumentException();
     this.length = length;
}
```

Η παράμετρος στη συνάρτηση setLength έχει ίδιο όνομα με τη μεταβλητή στιγμιότυπου length. Η παράμετρος λειτουργεί σαν τοπική μεταβλητή και αποκρύπτει τη μεταβλητή length του στιγμιότυπου. Το αναγνωριστικό length μέσα στην setLength αναφέρεται στην παράμετρο. Αν θέλουμε μέσα στην setLength να αναφερθούμε στη μεταβλητή του στιγμιότυπου μπορούμε να χρησιμοποιήσουμε το this όπως φαίνεται στον παραπάνω κώδικα.

Ανάλογα με τα συμφραζόμενα (context), η λέξη-κλειδί this, μπορεί επιπλέον να χρησιμοποιηθεί για την κλήση ενός δημιουργού της κλάσης μέσα σε έναν άλλο δημιουργό.

```
public Rectangle(double len) {
     this(length,length);
}
```

Εδώ ξαναγράφουμε τον δημιουργό κατασκευής τετραγώνων. Το τετράγωνο δημιουργείται με κλήση στον δημιουργό που έχει δύο παραμέτρους και επιτυγχάνεται με χρήση του this. Ας σημειωθεί πως τέτοια κλήση μπορεί να γίνει μόνο στην πρώτη γραμμή ενός δημιουργού.

11.5 Εμβέλεια και χρόνος ζωής

Τα στατικά μέλη της κλάσης φορτώνονται στη μνήμη συνήθως όταν η εφαρμογή χρειασθεί την κλάση στην οποία ανήκουν. Στη συνέχεια διατηρούνται μέχρι το τέλος της εφαρμογής. Επομένως, αυτά τα όρια διαμορφώνουν τον χρόνο ζωής τους. Αντίθετα, τα μέλη στιγμιότυπου δημιουργούνται ταυτόχρονα με το στιγμιότυπο στο οποίο ανήκουν και καταστρέφονται μαζί με αυτό.

Η εμβέλεια των μελών της κλάσης καθορίζεται από τον προσδιοριστή προσπέλασης του μέλους σε συνδυασμό με τον προσδιοριστή προσπέλασης της κλάσης. Για παράδειγμα, ένα μέλος που ανήκει σε δημόσια κλάση και είναι δημόσιο και το ίδιο μπορεί να προσπελαστεί από οποιονδήποτε χρήστη της κλάσης. Ένα μέλος όμως που ανήκει σε δημόσια κλάση αλλά είναι το ίδιο ιδιωτικό μπορεί να προσπελαστεί μόνο από την κλάση

στην οποία ανήκει. Παρόμοια, ένα μέλος που είναι δημόσιο αλλά ανήκει σε κλάση με πρόσβαση πακέτου μπορεί να προσπελαστεί μόνο από κλάσεις του ίδιου πακέτου.

11.6 Άλλες αναγκαίες συναρτήσεις της κλάσης

Προκειμένου η κλάση μας να είναι ολοκληρωμένη θα πρέπει να διαθέτει ως μέλη τρεις ακόμη συναρτήσεις, την toString, την equals και την hashCode.

11.6.1 Η συνάρτηση toString

Αν προσπαθήσουμε να εκτυπώσουμε ένα αντικείμενο της κλάσης Rectangle, θα λάβουμε ως έξοδο μια περίεργη εκ πρώτης όψεως αλφαριθμητική σειρά. Για παράδειγμα, ο ακόλουθος κώδικας

```
Rectangle k=new Rectangle(3, 6);
System.out.println(k);
```

θα τυπώσει κάτι σαν PackageName.Rectangle@4769b07b.
Πρόκειται για το όνομα της κλάσης, ακολουθούμενο από τον κωδικό κατακερματισμού του στιγμιότυπου k εκφρασμένο σε δεκαεξαδική μορφή. Στις περισσότερες περιπτώσεις όμως που επιχειρούμε να τυπώσουμε ένα στιγμιότυπο, εκείνο που θέλουμε είναι η τιμή του στιγμιότυπου και όχι το όνομα της κλάσης συνοδευόμενο από τον κωδικό κατακερματισμού. Η Java μας δίνει τη δυνατότητα να το πετύχουμε ορίζοντας κατάλληλα τη συνάρτηση toString().

```
@Override
public String toString() {
     return "Rectangle{" + "length=" + length + ", width=" +
     width + '}';
}
```

Η συνάρτηση toString είναι υποχρεωτικά public και επιστρέφει μια αναπαράσταση του στιγμιότυπου που την καλεί με τη μορφή αλφαριθμητικής σειράς. Επιπλέον, από τον ορισμό της συνάρτησης προηγείται η ειδική σημείωση (annotation) @Override. Η σημείωση αυτή δεν είναι υποχρεωτική από τον μεταγλωττιστή, ωστόσο είναι χρήσιμη και δεν θα πρέπει να ξεχνάμε να την προσθέτουμε. Το νόημά της σχετίζεται με την κληρονομικότητα και το παρουσιάζουμε στην ενότητα 13.1.1. Στο σώμα της συνάρτησης toString βάζουμε τον κώδικα που καθορίζει τι θα παίρνουμε σε όλες τις περιπτώσεις που χρησιμοποιούμε ένα στιγμιότυπο της κλάσης αλλά χρειάζεται μια αλφαριθμητική αναπαράστασή του. Έτσι τώρα, η εκτύπωση του αντικειμένου k θα τυπώσει Rectangle{length=3.0, width=6.0}. Η συνάρτηση toString() καλείται αυτόματα (implicitly) όπου είναι απαραίτητο, αλλά μπορεί να κληθεί και σαφώς (explicitly) από την εφαρμογή.

11.6.2 Η συνάρτηση equals

Μια άλλη αναγκαία συνάρτηση είναι η equals. Όπως έχουμε αναφέρει, τα στιγμιότυπα συνιστούν μεταβλητές αναφοράς. Επομένως, η σύγκριση μεταξύ δύο στιγμιότυπων με τον τελεστή == (equality operator) συγκρίνει τις διευθύνσεις τους και όχι τις τιμές τους. Έστω ο κώδικας

```
Rectangle r1=new Rectangle(2,6), r2=new Rectangle(2,6);
System.out.println(r1==r2);
```

Παρόλο που r1 και r2 είναι 2 ορθογώνια με ίδιες διαστάσεις, ο έλεγχος ισότητας εδώ μας επιστρέφει false. Πράγματι, έχουμε 2 διαφορετικά στιγμιότυπα. Η μεταβλητή r1 αναφέρεται στη μνήμη που έχει κατανεμηθεί για το ένα ορθογώνιο και η r2 στη μνήμη του άλλου. Επομένως, το αποτέλεσμα false είναι αναμενόμενο. Πώς όμως μπορούμε να συγκρίνουμε τα ίδια τα στιγμιότυπα και όχι τις διευθύνσεις τους; Εδώ έρχεται η συνάρτηση equals να δώσει τη λύση.

```
@Override
```

```
public boolean equals(Object obj) {
     if (this == obj) {
          return true;
     }
     if (obj == null) {
          return false;
     }
     if (getClass() != obj.getClass()) {
          return false;
     }
     final Rectangle other = (Rectangle) obj;
     if (Double.compare(width, other.width) != 0) {
          return false;
     }
     return Double.compare(length, other.length) == 0;
}
```

Η equals είναι αναγκαστικά public, επιστρέφει boolean και διαθέτει μια τυπική παράμετρο τύπου Object. Επιπλέον, η equals πρέπει να πληροί τις ακόλουθες ιδιότητες [1]:

1. Ανακλαστική: για οποιαδήποτε μη-μηδενική τιμή αναφοράς x, η έκφραση x.equals (x) πρέπει να επιστρέφει true.
2. Συμμετρική: για οποιεσδήποτε μη μηδενικές τιμές αναφοράς x και y, x.equals (y) πρέπει να επιστρέφει true εάν και μόνο εάν το y.equals (x) επιστρέφει true.
3. Μεταβατική: για οποιεσδήποτε μη μηδενικές τιμές αναφοράς x, y και z, εάν x.equals (y) επιστρέφει true και y.equals (z) επιστρέφει true, τότε x.equals (z) θα πρέπει να επιστρέψει true.
4. Για οποιεσδήποτε μη-μηδενικές τιμές αναφοράς x και y, όλες οι κλήσεις του x.equals (y) επιστρέφουν σταθερά true ή σταθερά false, υπό την προϋπόθεση ότι x και y παραμένουν σταθερά.
5. Για οποιαδήποτε μη-μηδενική τιμή αναφοράς x, x.equals (null) θα πρέπει να επιστρέφει false.

Μέσα στην equals αρχικά γίνεται έλεγχος αν το στιγμιότυπο που κάλεσε την equals είναι το ίδιο με την παράμετρό της. Σε αυτήν την περίπτωση επιστρέφεται true. Στη συνέχεια ελέγχεται αν η παράμετρος είναι null. Αν είναι, με δεδομένο ότι το στιγμιότυπο που κάλεσε την equals δεν είναι null, επιστρέφεται false. Μετά ελέγχεται αν η παράμετρος ανήκει στην ίδια κλάση με το στιγμιότυπο που κάλεσε την equals. Αν τα δύο αντικείμενα ανήκουν σε διαφορετικές κλάσεις, επιστρέφεται false. Στο επόμενο βήμα ελέγχονται οι μεταβλητές στιγμιότυπου των δύο αντικειμένων μια προς μια για ισότητα. Αν κάποιος από τους ελέγχους ισότητας αποτύχει, επιστρέφεται false. Τέλος, εφόσον η equals δεν έχει ήδη επιστρέψει, τα αντικείμενα θεωρούνται ίσα και επιστρέφεται true. Γενικά, μέσω της equals δίνουμε τον ορισμό ισότητας μεταξύ των αντικειμένων της κλάσης.

Όπως γνωρίζουμε, ο έλεγχος της ισότητας μεταξύ πραγματικών γίνεται συνήθως προσεγγιστικά, δηλαδή αν η απόλυτη τιμή της διαφοράς δύο πραγματικών είναι μικρότερη από μία οριακή τιμή που συνήθως ονομάζεται epsilon, οι αριθμοί θεωρούνται ίσοι. Ωστόσο, μια τέτοια προσεγγιστική ισότητα δεν πληροί τη μεταβατική ιδιότητα, επομένως η συνάρτηση equals δεν μπορεί να βασιστεί σε προσεγγιστική ισότητα μεταξύ των μελών της. Αν χρειάζεστε μια προσεγγιστική ισότητα, εφοδιάστε την κλάση σας με κατάλληλη συνάρτηση.

```
public boolean aproximateEquals(Rectangle r) {
     if (equals(r)) {
          return true;
     }
     if (Math.abs(width - r.width) < 0.001 && Math.abs(length
     - r.length) < 0.001) {
          return true;
     }
     return false;
}
```

Τέλος, ας σημειωθεί ότι ο τύπος της παραμέτρου της equals είναι Object. Ωστόσο, η πραγματική παράμετρος μπορεί να είναι Rectangle. Πρόκειται επίσης για ένα θέμα που σχετίζεται με την κληρονομικότητα και εξηγείται στην ενότητα 13.

11.6.3 Η συνάρτηση hashCode

Τα αντικείμενα της κλάσης μας ενδέχεται να φορτωθούν σε συλλογές κατακερματισμού (hash Collections), όπως το HashMap ή το HashSet. Οι συλλογές αυτές χρειάζονται ένα κλειδί κατακερματισμού (hash-key) για κάθε αντικείμενο που αποθηκεύουν το οποίο χρησιμοποιούν ώστε να επιτυγχάνουν ταχύτερη προσπέλαση. Καθώς κάθε αντικείμενο είναι πιθανό κάποια στιγμή να αποθηκευτεί σε συλλογή κατακερματισμού, θα πρέπει κάθε αντικείμενο να είναι σε θέση να υπολογίζει την τιμή κατακερματισμού του. Αυτός ακριβώς είναι ο σκοπός της συνάρτησης hashCode, να επιστρέψει μια τιμή κατακερματισμού για το αντικείμενο που την καλεί.

Η ανάπτυξη της hashCode πρέπει υποχρεωτικά να υπακούει σε τρεις κανόνες:

1. Κάθε κλήση της που γίνεται κατά τη διάρκεια της εκτέλεσης μιας εφαρμογής θα πρέπει να επιστρέφει το ίδιο hashCode για το ίδιο αντικείμενο. Μεταξύ διαφορετικών εκτελέσεων της ίδιας εφαρμογής δεν είναι απαραίτητο να επιστρέφεται ο ίδιος κωδικός κατακερματισμού.
2. Δύο αντικείμενα για τα οποία η equals επιστρέφει true θα πρέπει να έχουν τον ίδιο κωδικό κατακερματισμού
3. Δύο αντικείμενα για τα οποία η equals επιστρέφει false δεν είναι υποχρεωτικό να επιστρέφουν διαφορετικό κωδικό κατακερματισμού. Είναι όμως χρήσιμο. Στην περίπτωση για παράδειγμα που μια συνάρτηση hashCode επιστρέφει 1 για κάθε αντικείμενο μιας κλάσης, τότε η προσπέλαση μέσα στις συλλογές κατακερματισμού θα μετατραπεί σε σειριακή.

Γενικότερα, μία καλή συνάρτηση κατακερματισμού έχει ως στόχο την ελαχιστοποίηση παραγωγής ίδιων κωδικών για διαφορετικά αντικείμενα.
Ακολουθεί μια απλή συνάρτηση κατακερματισμού για την κλάση Rectangle.

```
@Override
public int hashCode() {
     return Double.hashCode(length) + Double.hashCode(width);
}
```

Γενικά μια συνάρτηση hashCode επιστρέφει έναν int, η τιμή του οποίου είναι συνάρτηση των τιμών των μεταβλητών του αντικειμένου που καλεί την hashCode. Εφόσον, οι μεταβλητές του αντικειμένου είναι αντικείμενα και οι ίδιες, μπορούμε να αξιοποιήσουμε τη συνάρτηση hashCode της κλάσης που ανήκει η μεταβλητή. Στην περίπτωσή που οι μεταβλητές είναι θεμελιώδεις (native) τύποι μπορούμε να χρησιμοποιήσουμε τη στατική hashCode της αντίστοιχης κλάσης όπως κάναμε σε αυτό το παράδειγμα.

Όπως αναφέραμε προηγουμένως, η συνάρτηση κατακερματισμού που παρουσιάζουμε είναι αρκετά απλή. Πληροί μεν τους κανόνες της συνάρτησης κατακερματισμού αλλά δεν είναι ιδιαίτερα αποτελεσματική. Για παράδειγμα, δύο αντικείμενα Rectangle, το ένα με μήκος 2 και πλάτος 6 και το άλλο με μήκος 6 και πλάτος 2, θα παράξουν τον ίδιο κωδικό κατακερματισμού παρότι είναι διαφορετικά, δηλαδή η equals επιστρέφει false.

Αυτό βέβαια το πρόβλημα μπορεί να αντιμετωπιστεί αν πολλαπλασιάσουμε τη μία διάσταση με έναν ακέραιο. Πάντως, όλα τα σύγχρονα IDE παράγουν αυτόματα τη συνάρτηση hashCode. Για παράδειγμα, το NetBeans, για την κλάση Rectangle, παραγάγει την ακόλουθη συνάρτηση κατακερματισμού:

```
@Override
public int hashCode() {
     int hash = 3;
     hash      =      53      *      hash      +      (int)
     (Double.doubleToLongBits(this.length)                  ^
     (Double.doubleToLongBits(this.length) >>> 32));
```

```
        hash = 53 * hash + (int)
        (Double.doubleToLongBits(this.width) ^
        (Double.doubleToLongBits(this.width) >>> 32));
        return hash;
    }
```

Η doubleToLongBits(length) επιστρέφει έναν long που όταν η δυαδική του αναπαράσταση μετατραπεί σε double, το αποτέλεσμα θα είναι ακριβώς ίσο με length. Στο αποτέλεσμα αυτό γίνεται μη προσημασμένη δεξιά διολίσθηση 32 θέσεων και αυτό που επιστρέφεται γίνεται αποκλειστικό or πάλι με την doubleToLongBits(length). Το νέο αποτέλεσμα το μετατρέπουμε σε int και προσθέτουμε έναν ακέραιο. Οι σταθερές 3 και 53, πιθανότατα να μην είναι ίδιες σε επαναλαμβανόμενες αιτήσεις προς το NetBeans να μας φτιάξει τη συνάρτηση κατακερματισμού της κλάσης Rectangle.

Περισσότερες πληροφορίες για το πώς μπορείτε να δημιουργείτε αποτελεσματικότερες συναρτήσεις κατακερματισμού μπορείτε να βρείτε στα βιβλία των Adekunle [2] και Aumasson, et all. [3].
Ακολουθεί η τελική έκδοση της κλάσης Rectangle στον κώδικα 11.8.

```
public class Rectangle {

    private double length, width;

    public Rectangle(double length, double width) {
        if (length <= 0 || width <= 0) {
            throw new IllegalArgumentException();
        }
        this.length = length;
        this.width = width;
    }

    public Rectangle(double length) {
        this(length, length);
    }

    public Rectangle() {
        this.length = 2;
        this.width = 2;
    }

    public double area() {
        return length * width;
    }

    public double perimeter() {
        return 2 * (this.length + this.width);
    }

    public double diagonal() {
        return Math.sqrt(Math.pow(this.length, 2) + Math.pow(this.width,
2));
    }

    public double getLength() {
        return this.length;
    }

    public void setLength(double length) {
        if (length <= 0) {
            throw new IllegalArgumentException();
        }
        this.length = length;
```